

KURA

RYO-CHAN Virtual Hackathon 2026 — Project Submission Form

Team

Total Members	1	
Member Name	Role	Email
[CONFIRM] full name as you want it credited	Builder / Sole Engineer	lydiasolomon137@gmail.com

Overview

Project	KURA — a deterministic pre-trade gate and tamper-evident decision ledger for agents built on RYO-CHAN.
Track	[CONFIRM] Select the matching track from #tracks. The executable verify_provenance skill points at the interoperability / skills track.
Problem	An autonomous trading agent is only as trustworthy as its behaviour when the data goes wrong. When an upstream research call is slow, incomplete or simulated, an LLM-driven agent reasons around the gap: it retries with a softer prompt, fills the hole with a plausible number, and emits a confident recommendation with no record of what it actually saw. Nothing about that is reproducible or auditable. RYO-CHAN publishes exactly the signals needed to catch this — status, data_mode, as_of, availability — and warns builders never to convert an unavailable measurement to zero. Most agents ignore those fields entirely.
Solution	KURA is an MCP gateway: MCP client upstream to RYO-CHAN, MCP server downstream to your agent. The agent registers KURA instead of the research tools, so it cannot route around the gate. Every candidate passes four synchronous TypeScript invariants with zero LLM calls . Approved candidates get a bounded fractional Kelly size; anything else halts with the exact failing gate and the empirical value that tripped it. Either way the decision is committed to a SHA-256 hash chain in WAL SQLite, and a standalone Python CLI recomputes that chain from raw disk state to prove nothing was altered.

Key Features	1. Zero-fallback invariant arbiter. Four gates in order, short-circuiting on first failure; fully synchronous, so it cannot hang on a degraded upstream. 270–300 μs cold, ~30 μs warm. • FRESHNESS — latencyMs <= 1200 && asOfAgeMs <= 300000 • ORACLE — status === 'ok'; partial and unavailable are refusals • PROVENANCE — data_mode === 'live'; refuses to size on simulated measurement • EVIDENCE — price and ATR(14) measurable; a null ATR is a veto, never a 0 2. Cryptographic flight recorder. H(n) = SHA256(H(n-1) : H_payload : timestamp : decision), genesis from 64 zeros, WAL SQLite, single IMMEDIATE transaction so no two blocks claim the same parent. 3. Independent provenance verifier. Standard-library Python sharing no code with the engine, ~0.06 ms/block. Required porting the ECMAScript Number::toString algorithm, since JSON.stringify(0.0000221) is "0.0000221" while Python repr gives '2.21e-05'. 4. Adversarial chaos console. Drop a connection, inject latency or force a malformed envelope from the browser and watch the breaker trip live. 5. MCP gateway on both transports. POST /mcp over Streamable HTTP, and npx kura gateway over stdio. 6. Live telemetry dashboard. SSE pushes tool latency, invariant state and committed ledger rows into a Next.js terminal with a per-receipt audit drawer. 7. Rate-limit aware transport. 429 and 503 retried with Retry-After honoured in both delta-seconds and HTTP-date form, falling back to exponential backoff with full jitter; 400 and 401 never retried, per the guide. Every backoff reports attempt, delay, reason, status and quota headers — a retry is never an untracked failure. Tested against a real local HTTP server that really returns 429. 8. Boot-time schema resolution probe. Reports which measurement paths resolved and raises SCHEMA RESOLUTION WARNING when RYO's data shape moves, instead of silently vetoing every token with a green test suite. 9. Empirical resilience benchmark. 150 real trials across three fault classes.
Target Users	Developers building autonomous or semi-autonomous trading agents on RYO-CHAN who need a pre-execution safety layer, and anyone who must later prove what their agent saw and decided — reviewers, auditors, or the builder's own post-mortem.
Scope	Research, evaluation, sizing recommendation and cryptographic record-keeping. Read-only by construction. RYO's surface cannot create or access a wallet, read balances or positions, or place, approve or execute a trade — and KURA adds no execution path of its own. There is no order-placement code, no signing key and no wallet anywhere in the repository. Position sizes are research output, not instructions.
Threat Model & Trust Assumptions	KURA is agent black-box auditing and deterministic circuit breaking, not a decentralised oracle. Stating the boundary precisely is what makes the guarantee worth anything. It proves: what the agent observed (the exact bytes RYO returned, hashed); when it observed them; that the invariants were evaluated before any downstream handoff (verdict and per-gate values sit in the same hashed block as the payload); and that the trail was not altered afterwards (each block links to its predecessor, verifiable by an independent tool). It does NOT prove: that RYO's data is correct — KURA can tell you a read was live and complete, not that it was right; that the recorded payload came from RYO — there is no response signature, so an operator who controls configuration could point the engine at a different MCP peer and mint a valid chain of fabricated observations; or that a downstream system obeyed the verdict — KURA has no execution path. Defends against: post-hoc tampering with the audit trail by anyone without the engine; silent upstream degradation (stale, incomplete or simulated data passing unnoticed); an operator's faulty recollection. Does not defend against: a malicious operator at configuration time; a compromised or impersonated RYO endpoint; a compromised host. Closing the gap needs response signing by RYO or anchoring block hashes to an external timestamping authority. Both require cooperation KURA cannot provide unilaterally, so neither is claimed.

Limitations	No live-endpoint run yet. Built and verified against the documented contract and the local conformance peer; the RYO_MCP_KEY had not arrived at submission time, so exact field names inside each tool's data block are unconfirmed. The signal extractor searches documented measurement names and vetoed on EVIDENCE if it cannot find them rather than guessing. GET /api/catalog surfaces the authenticated catalog for pinning exact paths. Sizing is a heuristic cap, not a Kelly proof. Real Kelly needs an estimated edge; KURA has none — it has “the evidence was complete and the asset was not too volatile”, a statement about data quality rather than expected return, with weights chosen for sane behaviour and not fitted to a backtest. The dashboard therefore leads with allocation as a percentage of the hard risk cap rather than a dollar figure. At or above 50% ATR/price it refuses to size at all rather than extrapolating past its own range. Single-process ledger. WAL SQLite on one node; a multi-writer deployment would need a different design. Not financial advice, and research output is not a guarantee of safety.
--------------------	--

Tech Stack

Frontend Stack	Next.js 14.2.18 (App Router), React 18.3, TypeScript 5.7, Tailwind CSS 3.4, Geist Sans + JetBrains Mono, native EventSource for SSE.
Backend Stack	Node.js 24 LTS, TypeScript 5.7, Fastify 5.2, @modelcontextprotocol/sdk 1.12 (client <i>and</i> server), Zod 3.23 runtime validation, better-sqlite3 12.2 in WAL mode, Vitest 2.1.
AI Model(s)	None in the decision path — deliberately. KURA calls no language model at any point during evaluation, sizing or verification; the arbiter is synchronous TypeScript. This is the core architectural claim and the test suite asserts it structurally: the gate is a non-async function, and 1,000 hostile-condition vetoes must average under 1 ms. Claude Code was used as a development tool to write the implementation.
Other Technologies	Python 3 standard library (hashlib, sqlite3, decimal) for the independent provenance verifier; SHA-256 hash chaining; Server-Sent Events; npm workspaces.

Repository / Demo

Github Repository	[ADD] private repo URL once the invite from @RYO Digital lands
Demo Video	[ADD] link once recorded — shot list in DEMO_SCRIPT.md
Documentation	README.md in the repository root — architecture, the four invariants, hash-chain construction, gateway tool reference, API surface and the resilience benchmark. SUBMISSION.md carries this form's content.

Testing Information

How to run the Project	<pre>git clone <repo-url> && cd KURA npm install cp .env.example .env npx kura start # engine + MCP gateway on :4000 npm run dev:web # dashboard on :3000</pre> Runs end to end with no credentials: .env.example defaults to the local conformance peer, a real MCP server over real stdio publishing the same six tools and the same public envelope as production. For live RYO-CHAN, set RYO_MCP_KEY and RYO_MCP_TRANSPORT=http. Fastest full walkthrough: <code>npm run demo</code>.
Prerequisites	Node.js 20+ (verified on 24.13.0). Python 3.8+ for the provenance verifier (verified on 3.14.7) — standard library only, no pip install. npm 9+ for workspaces. No database to provision: the SQLite file is created on first run. No API key required for the conformance path.

Installation Steps	<pre>git clone <repo-url> cd KURA npm install # installs both workspaces cp .env.example .env # runs against the conformance peer as-is</pre>
Environment Variables	All documented in <code>.env.example</code>, which ships with an empty credential. RYO_MCP_URL (endpoint, default <code>https://app-ryochan.com/api/mcp</code>) · RYO_MCP_KEY (builder credential, empty — you supply) · RYO_MCP_TRANSPORT (<code>http</code> <code>stdio</code>, default <code>stdio</code>) · RYO_RETRY_MAX_ATTEMPTS (4) · RYO_RETRY_BASE_DELAY_MS (500) · KELLY_HYPER_VOL_ATR_PCT (0.5) · EVIDENCE_PROBE (1) · ENGINE_PORT (4000) · LEDGER_PATH (<code>./kura_flight_recorder.db</code>) · INV_MAX_LATENCY_MS (1200) · INV_MAX_AS_OF_AGE_MS (300000) · KELLY_BANKROLL_USD (100000) · KELLY_FRACTION (0.25) · KELLY_MAX_POSITION_PCT (0.05) · KELLY_MAX_ATR_PCT (0.15) · PULSE_SWEEP_TOOLS (0) · KURA_WATCHLIST (SOL,BTC,ETH,AVAX,BNB). .env is gitignored; no credential is committed.
Build Command	<pre>npm run build # production Next.js build npm run typecheck # tsc --noEmit across both workspaces</pre>
Run Command	<pre>npx kura start # engine: REST + SSE + MCP gateway on :4000 npm run dev:web # dashboard on :3000 npx kura gateway # MCP gateway over stdio, for command-configured clients npx kura verify # walk and verify the entire ledger hash chain</pre>
Test Command	<pre>npm test # 103 tests, 8 files npm run test:resilience # 50 real trials per fault class python3 skills/verify_provenance/tool.py --all --quiet</pre> 103 tests pass across 8 files: canonical JSON and hash primitives; the six-tool surface and envelope validation; every gate at its exact boundary plus the sub-millisecond budget; WAL mode, genesis linkage and tamper localisation; the Python verifier reproducing TypeScript hashes across every number-formatting branch; a real MCP peer over real stdio (drop a tool → veto → SQLite commit → SSE delivery → verification → recovery); a real MCP client over real HTTP proving an external agent cannot route around the gate; and 429 with a real Retry-After, 503 falling back to full jitter, give-up behaviour and the codes that must not be retried, driven against a real local HTTP server. Resilience benchmark — 50 real trials per fault class, 100% detection and recovery: latency spike median 1.69 ms / p95 2.9 ms · malformed payload 0.47 ms / 0.93 ms · peer crash with real process respawn 519.42 ms / 595.21 ms.
Test Account(s)	None required. KURA has no user accounts, no login and no wallet — it is a local daemon plus a dashboard. The only credential is the RYO-CHAN builder key, and the default configuration runs the entire system without one. Conformance profiles usable as test inputs, each driving a distinct outcome: SOL APPROVED (clean, low ATR, larger size) · AVAX APPROVED (higher ATR, partial intelligence, smaller size) · STALE VETOED on FRESHNESS · PARTL VETOED on ORACLE (status "partial") · SIMUL VETOED on PROVENANCE (data_mode "simulated") · NOEVD VETOED on EVIDENCE (ATR(14) null) · BADEV SCHEMA_MISMATCH_OR_MISSING_FIELD (envelope missing data_mode).